Mac → Android · taller de 120 minutos

Una app, seis pasos de verdad.

Partimos de una pantalla vacía. Buscamos películas, las guardamos en el teléfono y construimos un catálogo propio. En cada fase copias archivos completos, pero entiendes qué cambió.

Antes de empezar

Necesitas una Mac con Android Studio, un emulador que ya funcione, conexión a internet y un editor como Visual Studio Code. No necesitas teléfono, Xcode ni otra imagen de Android si tu emulador ya abre.

> **Preparación previa recomendada:** comprueba permisos de instalación en la Mac de la escuela y solicita tu clave OMDb antes de la clase. La primera compilación descarga dependencias; 120 minutos no garantizan terminar instalaciones desde cero.

Solicita tu clave de películas

1. Abre OMDb · API Key. Elige la opción gratuita disponible, escribe tu correo y completa el formulario.
2. Revisa tu correo y spam. Abre el enlace de activación que envía OMDb.
3. Conserva tu clave. En la fase 1 la pondrás en `www/js/config.local.js`. No la pegues en el chat, GitHub ni capturas.
4. Si el correo no llega, puedes completar la pantalla y SQLite mientras esperas; para buscar películas sí necesitarás activarla.

Así se sigue cada fase

1. Abre únicamente la fase que indique el expositor.
2. Lee el mapa de archivos: crea los nuevos y reemplaza **todo** el contenido de los que cambiaron.
3. En VS Code: abre el archivo indicado → ⌘A → pega con ⌘V → guarda con ⌘S. Si es nuevo, créalo en la ruta exacta.
4. El fondo violeta y el signo **+** explican las diferencias. El botón copia el archivo completo, sin números ni marcas.
5. Ejecuta la prueba Android y compara el resultado antes de avanzar.

No descargues la última fase para empezar. Si te atrasas, recupera los archivos completos de **la fase actual**, sin tocar tu clave ni desinstalar la app.

> **El mapa mental**
> `www/index.html` es la pantalla; `www/css/index.css` su apariencia; `www/js/index.js` sus acciones; `www/js/db.js` habla con SQLite. Cordova los empaqueta dentro de una app Android.

Prepara la Mac, no la reinstales

Abre **Terminal** desde Spotlight. Estos primeros comandos consultan lo instalado. Si uno dice `command not found`, instala solo ese faltante.

Terminal · puedes ejecutarlo desde cualquier carpeta

```
node --version
npm --version
/usr/libexec/java_home -V
gradle --version
```

Referencia del taller: **Cordova CLI 13.0.0, Android 15.1.0 y SQLite 7.0.0**. Node 22.9 o superior dentro de la rama 22 es una opción compatible; Node 20.17+ también sirve para estas versiones. Usa **JDK 17** con Cordova, aunque Android Studio utilice otro Java.

▶ Me falta Node o Java 17

▶ Me falta Gradle

Conecta Terminal con Java y Android

En Android Studio abre **Tools → SDK Manager** (o **More Actions → SDK Manager** en la bienvenida). Confirma:

- **SDK Platforms:** Android 16 / API 36 para compilar.
- **SDK Tools:** Build-Tools 36.x, Platform-Tools, Command-line Tools (latest) y Android Emulator.
- Conserva la ruta de **Android SDK Location**. Normalmente es `~/Library/Android/sdk`.

API 36 es el SDK de compilación; no exige que tu dispositivo virtual tenga exactamente esa versión. Reutiliza el emulador existente.

Terminal · configura esta sesión; ajusta ANDROID_HOME si tu ruta es distinta

```
export JAVA_HOME="$(/usr/libexec/java_home -v 17)"
export ANDROID_HOME="$HOME/Library/Android/sdk"
export PATH="$JAVA_HOME/bin:$ANDROID_HOME/platform-tools:$ANDROID_HOME/cmdline-
tools/latest/bin:$ANDROID_HOME/emulator:$PATH"
java -version
adb --version
sdkmanager --version
```

Resultado: Java indica 17 y los otros dos comandos muestran su versión. Mantén esta Terminal abierta durante la práctica. Si abres otra, vuelve a ejecutar estas variables.

▶ Opcional: conservar las variables para futuras sesiones

Instala Cordova

Terminal · instalación de la herramienta

```
npm install -g cordova@13.0.0
cordova --version
```

`npm` descarga la herramienta; `-g` la deja disponible fuera de una carpeta particular. Espera que indique `13.0.0`. Si aparece `EACCES`, no añadas `sudo` a ciegas: revisa permisos de npm.

Primero: una app abierta en Android

En Terminal, crea una carpeta de trabajo nueva. Si ya existe `quever` en ella, elige otra carpeta vacía; no sobrescribas tu trabajo anterior.

Terminal · creación del taller en tu Escritorio

```
mkdir -p ~/Desktop/taller-cordova
cd ~/Desktop/taller-cordova
cordova create quever com.ejemplo.quever QueVer
cd quever
```

> **Qué acabamos de hacer**
> `create` genera la app. El primer `quever` es la carpeta; `com.ejemplo.quever` es el identificador Android; `QueVer` es el nombre visible. Para esta práctica todos pueden usarlo en emuladores separados.

Terminal · desde quever/; puede descargar dependencias

```
cordova platform add android@15.1.0
cordova plugin add cordova-sqlite-storage@7.0.0
cordova requirements android
```

Confirma que Java, SDK, target Android y Gradle aparecen disponibles. Si faltan licencias, ejecuta lo siguiente y revisa sus términos antes de aceptarlas:

Solo si hay licencias pendientes

```
sdkmanager --licenses
```

Abre **Android Studio → Device Manager** e inicia **un solo** dispositivo virtual con ▶. Espera a que muestre la pantalla de Android. No necesitas abrir el proyecto en Android Studio.

Terminal · desde quever/

```
cordova run android --emulator
```

> **No avances hasta ver esto**
>
> La app inicial de Cordova abre en el emulador. La primera compilación puede descargar Gradle y bibliotecas; las siguientes suelen ser más rápidas.

Abre `Desktop/taller-cordova/quever` con **VS Code → File → Open Folder**. Editaremos `www/` y `config.xml`, nunca los archivos generados de `platforms/android/`.

> Después de **cada fase**: guarda todos los archivos y ejecuta de nuevo `cordova run android --emulator` desde esta misma carpeta. No basta con refrescar una pestaña ni abrir `www/index.html` en Safari.

Una pantalla conectada al dispositivo

Al terminar tendrás una pantalla QueVer y una base local lista. Todavía no habrá buscador ni películas.

> **Para entender este bloque**
>
> Android abre una **WebView** con nuestro HTML. `cordova.js` conecta JavaScript con el contenedor. Esperamos `deviceready` antes de llamar al plugin SQLite. `initDB()` crea la tabla `movies` solo si no existe; por eso volver a abrir la app no borra tus datos.

Crea o reemplaza estos archivos

La ruta `www/` empieza dentro de tu carpeta `quever`. En `www/js/config.local.js`, sustituye `TU_API_KEY` por tu clave OMDb entre comillas. Si aún no llegó, deja el marcador por ahora.

Comprueba la fase 1

Guarda y ejecuta `cordova run android --emulator`.

1. Ves "QueVer" y "La base de tu catálogo está lista".
2. El estado cambia a "Tu catálogo está listo para recibir películas". Esto confirma que terminó la creación de la tabla.
3. Cierra y vuelve a abrir la app: el mensaje listo reaparece.

Todavía no: buscar, guardar, editar ni recomendar. Creamos el esquema completo de datos para evitar migraciones durante la clase, pero las operaciones llegarán cuando las usemos.

Del título a los resultados

Al terminar podrás buscar **Inception** y abrir el detalle de una película. Activa tu clave OMDb antes de probar.

Recorre el dato
El formulario dispara `submit` → `searchMovies()` lee el título → `requestOmdb()` usa `fetch` por HTTPS → OMDb devuelve JSON → `renderSearchResults()` crea los elementos de la pantalla. "Ver detalle" consulta el identificador IMDb. Esta búsqueda es web; SQLite todavía no guarda el resultado.

Conserva `www/js/config.local.js`. No pegues la plantilla otra vez. Si no había clave, abre ahora ese archivo y sustituye solo el marcador.

Comprueba la fase 2

Guarda y ejecuta `cordova run android --emulator`.

1. Busca **Inception**; deben aparecer resultados. Esto también comprueba tu clave y su activación.
2. Pulsa "Ver detalle": aparecen título, año y sinopsis.
3. Busca sin escribir: aparece un aviso para introducir un título.
4. Si OMDb dice "Invalid API key", vuelve al correo de activación y verifica tu archivo de clave.

Todavía no: guardar películas ni opiniones. El detalle explica que el guardado llega en la siguiente fase.

Tu primera película guardada

Ahora decides si quieres verla o ya la viste. Una vista puede llevar calificación y opinión.

De internet al almacenamiento local

El detalle trae datos de OMDb → `saveMovie()` añade estado y opinión → `insertMovie()` envía SQL al plugin → SQLite guarda el registro → consultamos los datos y actualizamos la lista. Los `?` de SQL son parámetros: separan la consulta de los valores. `imdb_id` es único para impedir duplicados.

Comprueba la fase 3

Guarda y ejecuta `cordova run android --emulator`.

1. Busca una película, abre el detalle y pulsa "La quiero ver". Debe aparecer en el listado.
2. Guarda otra con "Ya la vi", calificación y opinión.
3. Intenta guardar la primera otra vez: debe avisar que ya existe, sin sobrescribirla.
4. Cierra y vuelve a abrir QueVer desde Android. Las películas deben seguir guardadas.

Todavía no: catálogo por tarjetas, editar ni eliminar. No desinstales la app: eso sí puede borrar su base.

Los mismos datos, una vista más útil

Separamos películas pendientes y vistas, añadimos tarjetas y abrimos el detalle de lo guardado.

No estamos creando otra base

`getMoviesByStatus()` lee los registros de SQLite. `loadCatalog()` pide los dos estados y dibuja cada sección. La portada muestra hasta cinco por estado y "Ver más" muestra todos los registros ya consultados; no descarga otra página de OMDb.

Comprueba la fase 4

Guarda y ejecuta `cordova run android --emulator`.

1. Las películas de la fase anterior siguen ahí, separadas por estado.
2. Abre una tarjeta: ves su detalle guardado y, cuando corresponde, tu opinión.
3. Si tienes más de cinco en un estado, "Ver más" permite ver el resto. Con menos, no debe aparecer ese enlace.

Todavía no: modificar o borrar. Cambiamos la presentación, no la identidad de los registros.

El catálogo ya se puede mantener

Corrige opiniones, cambia de pendiente a vista o elimina una película con confirmación.

> **CRUD en nuestra app**
> Create: `INSERT`; Read: `SELECT`; Update: `UPDATE`; Delete: `DELETE`. El identificador IMDb dice qué registro cambiar. Primero esperamos el resultado de SQLite y después mostramos el éxito o actualizamos las tarjetas.

Comprueba la fase 5

Guarda y ejecuta `cordova run android --emulator`.

1. Abre una película vista y edita su opinión o calificación. Al volver a abrirla, conserva el cambio.
2. Marca una pendiente como vista: cambia de sección.
3. Pulsa eliminar y cancela: la película permanece.
4. Repite y confirma: desaparece. Reinicia la app y comprueba el estado final.

Todavía no: recomendación. El CRUD ya está completo.

Una nueva entrada, el mismo guardado

El botón “Recomiéndame una” busca una película en OMDb y permite guardarla con las acciones que ya conoces.

> **No es inteligencia artificial**
> Elegimos una palabra de búsqueda al azar, consultamos OMDb y elegimos un resultado. Pedimos su detalle antes de abrir el cuadro. `saveRandomMovie()` reutiliza `saveMovie()`; no inventamos otra base ni otro guardado.

Comprueba la fase 6

Guarda y ejecuta `cordova run android --emulator`.

1. Pulsa “Recomiéndame una”: aparece un título, sus datos y opciones de guardado.
2. Guárdala como pendiente o vista. Debe aparecer en el catálogo.
3. Cierra otra recomendación sin guardar: el catálogo no cambia.
4. Sin internet, la consulta avisa del fallo en lugar de mostrar una recomendación vacía.

Demuestra que es tu app

1. Busca una película y guárdala.
2. Edita su opinión y cierra la app desde la vista de aplicaciones recientes de Android.
3. Ábrela otra vez desde su icono. La película y tu cambio siguen ahí.
4. Sin red, comprueba que puedes consultar lo ya guardado; buscar y recomendar sí necesitan internet.
5. Explica: ¿qué parte ejecuta la WebView?, ¿qué hace el plugin?, ¿dónde viven los datos?

> **Lo que construiste**
>
> Una app Android con interfaz HTML/CSS/JavaScript, consulta HTTP a OMDb y almacenamiento SQLite nativo mediante Cordova.

▶ Opcional: instalarla en tu teléfono Android

Si algo falla, vuelve a este punto

▶ **¿En qué carpeta ejecuto Cordova?**

▶ **No reconoce node, npm o cordova / error EACCES**

▶ **Java incorrecto o ANDROID_HOME no encontrado**

▶ **Falta target Android, Build-Tools, sdkmanager o licencias**

▶ **No aparece el emulador o hay varios destinos**

▶ **Gradle descarga mucho, tarda o falla**

▶ **SQLite no disponible o pantalla “Preparando…”**

▶ **La búsqueda falla o dice “Invalid API key”**

▶ **Guardé archivos pero la pantalla no cambia / me atrasé**

▶ **Quiero inspeccionar el error real**